```
> ##############################################################
  #  TauTimp_Spherical_Wave_TimeInt.mpl
  #  TauTimp — Variable-Coefficient Spherical Wave Equation
  #  + Probability Density  d^2P/dt^2 - c^2*d^2P/dr^2 = 0
  #  Initial: P(r,0) = A*r^2+B*r+C  with BC P(R,0)=0
  #  Condition: int(P(R,t), t=0.290Gy..13.79Gy) = 47 Gly
  #  Author: Viorel Popescu — TauTimp Project 2026
  #  Execute: !!! (Execute All) in Maple 2022+
  ##############################################################

  restart;
  with(plots): with(plottools): with(LinearAlgebra):

  ##############################################################
  # SECTION 0 — Physical constants and parameters
  ##############################################################
  printf("=== SECTION 0 : Physical constants ===\n");

  c    := 2.99792458e8:           # speed of light    [m/s]
  hbar := 1.054571817e-34:        # hbar              [J•s]
  R    := 4.235518529e22:              # boundary param    [m]
  Pi_  := evalf(Pi):

  s_per_Gy := 3.15576e16:         # 1 Gy in seconds
  ly_to_m  := 9.4607304725808e15:
  Gly_to_m := ly_to_m * 1e9:

  # Time limits and target
  t1 := evalf(0.290e9 * s_per_Gy):   # 0.290 Gy in seconds
  t2 := evalf(13.79e9 * s_per_Gy):   # 13.79 Gy in seconds
  target := evalf(47 * Gly_to_m):    # 47 Gly in meters

  # Airy zeros  a_n  (n-th zero of Ai with negated argument)
  a1 := 2.338107411:
  a2 := 4.087949444:
  a3 := 5.520559828:

  # Eigenvalues  lambda_n = a_n^{3/2} / R,  omega_n = lambda_n * c
  lam1 := evalf(a1^(3/2) / R):
  lam2 := evalf(a2^(3/2) / R):
  lam3 := evalf(a3^(3/2) / R):
  om1  := evalf(c * lam1):
```

```
om2  := evalf(c * lam2):
om3  := evalf(c * lam3):

# Time integral  int(t^2, t1..t2) = (t2^3-t1^3)/3
t_sq_int := evalf((t2^3 - t1^3)/3):

# Coefficient A from normalisation condition
A_coef := evalf(target / (c^2 * t_sq_int)):

# Canonical choice: P(r,0)=A*(r-R)^2  =>  B=-2AR, C=AR^2
B_coef := evalf(-2 * A_coef * R):
C_coef := evalf(A_coef * R^2):

printf("  c        = %g m/s\n",     c);
printf("  R        = %g m\n",       R);
printf("  t1       = %g s  (0.290 Gy)\n", t1);
printf("  t2       = %g s  (13.79 Gy)\n", t2);
printf("  target   = %g m  (47 Gly)\n",   target);
printf("  int(t^2) = %g s^3\n",     t_sq_int);
printf("  a1=%g  a2=%g  a3=%g\n",   a1, a2, a3);
printf("  lam1=%g  lam2=%g  lam3=%g  [m^-1]\n", lam1, lam2, lam3)
;
printf("  om1 =%g  om2 =%g  om3 =%g  [rad/s]\n", om1,  om2,  om3)
;
printf("  A   = %g  m^-1 s^-1\n",  A_coef);
printf("  B   = %g  s^-1\n",       B_coef);
printf("  C   = %g  m s^-1\n",     C_coef);

################################################################
# SECTION 1 — Symbolic PDE for f(r,t) — wave equation
################################################################
printf("\n=== SECTION 1 : Symbolic wave PDE ===\n");

f_sym := f(r, t):
PDE_f := (R/r)*(diff(f_sym,r,r) + (2/r)*diff(f_sym,r))
       - (1/c^2)*diff(f_sym,t,t) = 0;

printf("  Master PDE: (R/r)*(f_rr+2/r*f_r) = (1/c^2)*f_tt\n");
print(PDE_f);

################################################################
# SECTION 2 — Separation of variables f = g(r)*u(t)
```

```
#################################################################
printf("\n=== SECTION 2 : Separation of variables ===\n");

printf("  ODE (T): u'' + lambda^2*c^2*u = 0\n");
printf("  ODE (R): g'' + (2/r)*g' + (lambda^2*r/R)*g = 0\n\n");

# Temporal solution
ODE_T := diff(u(t),t,t) + lam1^2*c^2*u(t) = 0;
sol_T := dsolve(ODE_T, u(t));
printf("  u(t) [n=1]: "); print(sol_T);

printf("  Radial: sub g=v/r => Airy eq v''+(lam^2*r/R)*v=0\n");
printf("  xi = -(lam^2/R)^{1/3}*r  =>  v'' - xi*v = 0\n");
printf("  Solution: g(r) = (1/r)*AiryAi(xi)\n");

#################################################################
# SECTION 3 — Airy eigenfunctions and BC quantisation
#################################################################
printf("\n=== SECTION 3 : Eigenvalue quantisation  f(R,0)=0 ===
\n");

xi_n := proc(rr, lam)  -(lam^2/R)^(1/3)*rr  end proc:
g_n  := proc(rr, lam)  AiryAi(xi_n(rr,lam)) / rr  end proc:

g1 := proc(rr)  g_n(rr, lam1)  end proc:
g2 := proc(rr)  g_n(rr, lam2)  end proc:
g3 := proc(rr)  g_n(rr, lam3)  end proc:

printf("  BC f(R,0)=0 => Ai(-(lam_n^2/R)^{1/3}*R)=0\n");
printf("  => lam_n = a_n^{3/2}/R\n\n");
for n from 1 to 3 do
    an_  := [a1,a2,a3][n]:
    lamn := an_^(3/2)/R:
    val  := evalf(AiryAi(xi_n(R, lamn))):
    printf("  n=%d  a_n=%.6f  lam_n=%g  Ai(xi(R))=%g\n", n, an_,
lamn, val);
end do;

#################################################################
# SECTION 4 — Full f(r,t) and H(r,theta,t)
#################################################################
printf("\n=== SECTION 4 : Full field f(r,t) and H(r,theta,t) ===
```

```
\n");

printf("  f(r,t) = SUM_n (1/r)*Ai(-(lam_n^2/R)^{1/3}*r)*[A_n cos
+ B_n sin]\n");
printf("  f_1(R,  0) = %g  (BC check ~0)\n", evalf(g1(R)));
printf("  f_1(2R, 0) = %g\n",                evalf(g1(2*R)));

printf("\n  H(r,theta,t) = SUM_{n,l} g_n(r)*P_l(cos theta)*
[cos/sin]\n");
printf("  P_0=%g  P_1=%g  P_2=%g  at theta=Pi/4\n",
       evalf(LegendreP(0,cos(Pi_/4))),
       evalf(LegendreP(1,cos(Pi_/4))),
       evalf(LegendreP(2,cos(Pi_/4))));

################################################################
# SECTION 5 — Probability density PDE and d'Alembert solution
################################################################
printf("\n=== SECTION 5 : Probability density PDE ===\n");

P_sym := P(r, t):
PDE_P := diff(P_sym,t,t) - c^2*diff(P_sym,r,r) = 0;
printf("  Probability PDE: d^2P/dt^2 - c^2*d^2P/dr^2 = 0\n");
print(PDE_P);

printf("  d'Alembert: P(r,t) = phi+(r+ct) + phi-(r-ct)\n");
printf("  With P(r,0)=A*r^2+B*r+C and dP/dt|0=0:\n");
printf("  => phi+ = phi- = (A*r^2+B*r+C)/2\n");
printf("  => P(r,t) = A*(r^2+c^2*t^2) + B*r + C  [exact!]\n");

################################################################
# SECTION 6 — Analytic determination of A, B, C
################################################################
printf("\n=== SECTION 6 : Analytic determination of A, B, C ===
\n");

# At r=R with BC A*R^2+B*R+C=0:
# P(R,t) = A*(R^2+c^2*t^2)+B*R+C = A*c^2*t^2 + (A*R^2+B*R+C) = A*
c^2*t^2
# int_t1^t2 P(R,t) dt = A*c^2 * (t2^3-t1^3)/3 = 47 Gly

printf("  P(R,t) = A*c^2*t^2  [using BC: A*R^2+B*R+C=0]\n");
printf("  int(P(R,t),t=t1..t2) = A*c^2*(t2^3-t1^3)/3\n");
```

```
printf("  => A = 47Gly / [c^2*(t2^3-t1^3)/3]\n\n");

printf("  t2^3-t1^3 = %g s^3\n",   evalf(t2^3-t1^3));
printf("  (t2^3-t1^3)/3 = %g s^3\n", t_sq_int);
printf("  c^2*(t2^3-t1^3)/3 = %g m^2*s\n", evalf(c^2*t_sq_int));
printf("\n  A = %g  m^-1 s^-1\n", A_coef);
printf("  B = -2*A*R = %g  s^-1\n", B_coef);
printf("  C = A*R^2  = %g  m s^-1\n", C_coef);

# Verification
BC_check   := evalf(A_coef*R^2 + B_coef*R + C_coef):
integ_check := evalf(A_coef * c^2 * t_sq_int):
printf("\n  VERIFICATION:\n");
printf("  BC: A*R^2+B*R+C = %g  (->0)\n",    BC_check);
printf("  Integral = %g m\n",                 integ_check);
printf("  Target   = %g m  (47 Gly)\n",       target);
printf("  Ratio    = %g  (->1)\n",            integ_check/target)
;
printf("  A != 0 : %a\n", evalb(abs(A_coef) > 0));
printf("  C != 0 : %a\n", evalb(abs(C_coef) > 0));

################################################################
# SECTION 7 — Explicit probability density and flux
################################################################
printf("\n=== SECTION 7 : Explicit P(r,t) and flux F(r,t) ===\n")
;

# P(r,t) = A*((r-R)^2 + c^2*t^2)
P_full := proc(rr, tt)
    A_coef * ((rr-R)^2 + c^2*tt^2)
end proc:

F_flux := proc(rr, tt)
    P_full(rr, tt) / (4*Pi_*rr^2)
end proc:

printf("  P(r,t) = A*[(r-R)^2 + c^2*t^2]\n");
printf("  F(r,t) = P(r,t)/(4*Pi*r^2)\n\n");
printf("  P(R,  0)  = %g  (BC check)\n",   evalf(P_full(R,0)));
printf("  P(2R, 0)  = %g\n",               evalf(P_full(2*R,0)));
printf("  P(R, t1)  = %g m/s\n",           evalf(P_full(R,t1)));
printf("  P(R, t2)  = %g m/s\n",           evalf(P_full(R,t2)));
```

```
printf("  dP/dt|t=0 = %g  (should be 0)\n", evalf(2*A_coef*c^2*0)
);

#################################################################
# SECTION 8 — 12 Structured Plots
#################################################################
printf("\n=== SECTION 8 : Generating 12 plots ===\n");

# ---- Plot 1 : Airy functions Ai(xi) and Bi(xi) ----
p1 := plot([AiryAi(x), AiryBi(x)],
           x = -9..2,
           color     = [blue, red],
           linestyle = [solid, dash],
           legend    = ["Ai(xi)","Bi(xi)"],
           title     = "Plot 1 — Airy Functions Ai(xi) and Bi
(xi)",
           labels    = ["xi","value"],
           view      = [-9..2,-1.5..1.5],
           thickness = 2,
           gridlines = true):
printf("  Plot 1 done\n");

# ---- Plot 2 : Radial eigenfunctions g_n(r) ----
r_lo := 0.01*R:  r_hi := 3*R:
p2 := plot([g1, g2, g3],
           r_lo..r_hi,
           color     = [blue, red, black],
           legend    = ["n=1","n=2","n=3"],
           title     = "Plot 2 — Radial Eigenfunctions g_n(r) =
Ai(xi_n)/r",
           labels    = ["r  [m]","g_n"],
           thickness = 2,
           gridlines = true):
printf("  Plot 2 done\n");

# ---- Plot 3 : Temporal oscillations u_n(t) ----
T1 := evalf(2*Pi_/om1):
p3 := plot([proc(tt) cos(om1*tt) end proc,
            proc(tt) cos(om2*tt) end proc,
            proc(tt) cos(om3*tt) end proc],
           0..2*T1,
           color     = [blue, red, black],
```

```
           legend    = ["n=1","n=2","n=3"],
           title     = "Plot 3 — Temporal Harmonics cos(omega_n*
t)",
           labels    = ["t  [s]","u_n"],
           thickness = 2,
           gridlines = true):
printf("  Plot 3 done\n");

# ---- Plot 4 : P(r,0) = A*(r-R)^2  vs  r ----
P_t0 := proc(rr)  A_coef*(rr-R)^2  end proc:
p4 := plot(P_t0,
           0..5*R,
           color     = blue,
           thickness = 2,
           title     = "Plot 4 — Initial Density P(r,0) = A*(r-
R)^2",
           labels    = ["r  [m]","P(r,0)  [m/s]"],
           gridlines = true):
printf("  Plot 4 done\n");

# ---- Plot 5 : P(R,t) = A*c^2*t^2  vs  t (in Gy) ----
P_Rt := proc(tt)  A_coef*c^2*tt^2  end proc:
p5 := plot(P_Rt,
           0..t2,
           color     = blue,
           thickness = 2,
           title     = "Plot 5 — Density at Boundary P(R,t) = A*
c^2*t^2",
           labels    = ["t  [s]","P(R,t)  [m/s]"],
           gridlines = true):
printf("  Plot 5 done\n");

# ---- Plot 6 : Cumulative integral int(P(R,t),0..t) ----
P_cum := proc(tt)  evalf(A_coef*c^2*tt^3/3)  end proc:
p6 := plot(P_cum,
           t1..t2,
           color     = blue,
           thickness = 2,
           title     = "Plot 6 — Cumulative int(P(R,t),t1..t)
[m]",
           labels    = ["t  [s]","Cumulative integral  [m]"],
           gridlines = true):
```

```
printf("  Plot 6 done\n");

# ---- Plot 7 : Flux F(r,0) = A*(r-R)^2/(4*Pi*r^2) ----
F_t0 := proc(rr)  A_coef*(rr-R)^2/(4*Pi_*rr^2)  end proc:
p7 := plot(F_t0,
           0.1*R..8*R,
           color     = blue,
           thickness = 2,
           title     = "Plot 7 — Flux F(r,0) = P(r,0)/(4*Pi*r^2)
",
           labels    = ["r  [m]","F(r,0)  [s^{-1}]"],
           gridlines = true):
printf("  Plot 7 done\n");

# ---- Plot 8 : 2D density P(r,t) in (r,t) plane ----
r_scale := R:  t_scale := t1:   # use t1 as unit for time axis
P_2D := proc(rr, tt)
    evalf(A_coef*((rr*r_scale - R)^2 + c^2*(tt*t_scale)^2))
end proc:

p8 := densityplot(proc(rr,tt) P_2D(rr,tt) end proc,
                  0.5..5,
                  0..10,
                  grid        = [50,50],
                  colorscheme = ["zgradient",["navy","blue",
"cyan","yellow","red"]],
                  title       = "Plot 8 — P(r,t) Density  [r in
R-units, t in t1-units]",
                  labels      = ["r/R","t/t1"]):
printf("  Plot 8 done\n");

# ---- Plot 9 : 3D surface P(r,t) ----
p9 := plot3d(proc(rr,tt) P_2D(rr,tt) end proc,
             0.5..4,
             0..8,
             grid        = [30,30],
             orientation = [45,65],
             colorscheme = ["zgradient",["navy","cyan","yellow",
"red"]],
             title       = "Plot 9 — 3D Surface P(r,t)  [r in R,
t in t1]",
             labels      = ["r/R","t/t1","P"]):
```

```
printf("  Plot 9 done\n");

# ---- Plot 10 : 2D density H(r,theta) for (n=1,l=0) at t=0 ----
H2D := proc(rr, th)
    AiryAi(xi_n(rr,lam1))/rr * LegendreP(0,cos(th))
end proc:

p10 := densityplot(proc(rr,th) H2D(rr,th) end proc,
                   r_lo..2.5*R,
                   0..Pi_,
                   grid        = [50,40],
                   colorscheme = ["zgradient",["blue","white",
"red"]],
                   title       = "Plot 10 — 2D Density H(r,
theta,0) [n=1,l=0]",
                   labels      = ["r  [m]","theta  [rad]"]):
printf("  Plot 10 done\n");

# ---- Plot 11 : Eigenfrequency spectrum omega_n ----
p11 := pointplot([[1,om1],[2,om2],[3,om3]],
                 symbol     = solidcircle,
                 symbolsize = 20,
                 color      = blue,
                 title      = "Plot 11 — Eigenfrequencies
omega_n vs n",
                 labels     = ["n","omega_n  [rad/s]"],
                 gridlines  = true):
printf("  Plot 11 done\n");

# ---- Plot 12 : 4-Panel Summary ----
p_A := plot([AiryAi(x),AiryBi(x)], x=-8..2,
            color=[blue,red], view=[-8..2,-1.5..1.5],
            title="(a) Airy Ai, Bi", labels=["xi",""], thickness=
2):

p_B := plot([g1,g2,g3], r_lo..2.5*R, color=[blue,red,black],
            title="(b) Radial g_n(r)", labels=["r","g_n"],
thickness=2):

p_C := plot(P_t0, 0.1*R..5*R, color=blue,
            title="(c) P(r,0)=A*(r-R)^2", labels=["r","P"],
thickness=2):
```

```
p_D := plot(F_t0, 0.5*R..6*R, color=blue,
            title="(d) Flux F(r,0)", labels=["r","F"], thickness=
2):

p12 := display(Array([[p_A,p_B],[p_C,p_D]]),
               title="Plot 12 — 4-Panel Summary"):
printf("  Plot 12 done\n");

################################################################
# Display all plots
################################################################
printf("\n=== Displaying plots 1-12 ===\n");
display(p1);  display(p2);  display(p3);
display(p4);  display(p5);  display(p6);
display(p7);  display(p8);  display(p9);
display(p10); display(p11); display(p12);

################################################################
# SECTION 9 — Summary table
################################################################
printf("\n=== SECTION 9 : Complete Results Summary ===\n");
printf("-----------------------------------------------------------
-----------\n");
printf("  Quantity                                  Value\n");
printf("-----------------------------------------------------------
-----------\n");
printf("  R  (boundary parameter)                  %g m\n",       R)
;
printf("  c  (speed of light)                      %g m/s\n",     c)
;
printf("  t1  = 0.290 Gy                           %g s\n",
t1);
printf("  t2  = 13.79 Gy                           %g s\n",
t2);
printf("  47 Gly                                   %g m\n",
target);
printf("  int(t^2,t1..t2) = (t2^3-t1^3)/3      %g s^3\n",
t_sq_int);
printf("  c^2*(t2^3-t1^3)/3                        %g m^2*s\n",
evalf(c^2*t_sq_int));
printf("  Airy zeros  a1,a2,a3                     %g  %g  %g\n",a1,
a2,a3);
```

```
  printf("  lambda1=a1^{3/2}/R                    %g m^-1\n",
  lam1);
  printf("  lambda2=a2^{3/2}/R                    %g m^-1\n",
  lam2);
  printf("  lambda3=a3^{3/2}/R                    %g m^-1\n",
  lam3);
  printf("  omega1=c*lam1                         %g rad/s\n",
  om1);
  printf("  omega2=c*lam2                         %g rad/s\n",
  om2);
  printf("  omega3=c*lam3                         %g rad/s\n",
  om3);
  printf("  A = 47Gly/(c^2*(t2^3-t1^3)/3)        %g m^-1 s^-1\n",
  A_coef);
  printf("  B = -2*A*R                            %g s^-1\n",
  B_coef);
  printf("  C = A*R^2                             %g m s^-1\n",
  C_coef);
  printf("  BC check A*R^2+B*R+C                  %g  (->0)\n",
  evalf(A_coef*R^2+B_coef*R+C_coef));
  printf("  Integral A*c^2*(t2^3-t1^3)/3          %g m\n",
  evalf(A_coef*c^2*t_sq_int));
  printf("  Target 47 Gly                         %g m\n",
  target);
  printf("  dP/dt|t=0                             0  (rest IC)\n");
  printf("  P(r,0) form                           A*(r-R)^2\n");
  printf("  P(r,t) form                           A*[(r-R)^2+c^2*
  t^2]\n");
  printf("------------------------------------------------------
  ----------\n");
  printf("\nAll 9 sections and 12 plots completed.\n");
  printf("TauTimp_Spherical_Wave_TimeInt.mpl — Viorel Popescu
  2026\n");

=== SECTION 0 : Physical constants ===
  c        = 2.99792e+08 m/s
  R        = 4.23552e+22 m
  t1       = 9.15170e+24 s  (0.290 Gy)
  t2       = 4.35179e+26 s  (13.79 Gy)
  target   = 4.44654e+26 m  (47 Gly)
  int(t^2) = 2.74713e+79 s^3
  a1=2.33811  a2=4.08795  a3=5.52056
  lam1=8.44093e-23  lam2=1.95142e-22  lam3=3.06244e-22  [m^-1]
```

om1 =2.53053e-14 om2 =5.85022e-14 om3 =9.18097e-14 [rad/s]
A   = 1.80095e-70 m^-1 s^-1
B   = -1.52559e-47 s^-1
C   = 3.23084e-25 m s^-1

=== SECTION 1 : Symbolic wave PDE ===

$$PDE_f := \frac{4.235518529 \times 10^{22} \left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2 \left(\frac{\partial}{\partial r} f(r,t) \right)}{r} \right)}{r} - 1.112650056 \times 10^{-17} \frac{\partial^2}{\partial t^2} f(r,t) = 0$$

Master PDE: (R/r)*(f_rr+2/r*f_r) = (1/c^2)*f_tt

$$\frac{4.235518529 \times 10^{22} \left(\frac{\partial^2}{\partial r^2} f(r,t) + \frac{2 \left(\frac{\partial}{\partial r} f(r,t) \right)}{r} \right)}{r} - 1.112650056 \times 10^{-17} \frac{\partial^2}{\partial t^2} f(r,t) = 0$$

=== SECTION 2 : Separation of variables ===
ODE (T): u'' + lambda^2*c^2*u = 0
ODE (R): g'' + (2/r)*g' + (lambda^2*r/R)*g = 0

$$ODE_T := \frac{d^2}{dt^2} u(t) + 6.403561864 \times 10^{-28} u(t) = 0$$

$$sol_T := u(t) = c_1 \sin\left(\frac{7\sqrt{81678085}\, t}{2500000000000000000} \right) + c_2 \cos\left(\frac{7\sqrt{81678085}\, t}{2500000000000000000} \right)$$

u(t) [n=1]:

$$u(t) = c_1 \sin\left(\frac{7\sqrt{81678085}\, t}{2500000000000000000} \right) + c_2 \cos\left(\frac{7\sqrt{81678085}\, t}{2500000000000000000} \right)$$

Radial: sub g=v/r => Airy eq v''+(lam^2*r/R)*v=0
xi = -(lam^2/R)^{1/3}*r => v'' - xi*v = 0
Solution: g(r) = (1/r)*AiryAi(xi)

=== SECTION 3 : Eigenvalue quantisation f(R,0)=0 ===
BC f(R,0)=0 => Ai(-(lam_n^2/R)^{1/3}*R)=0
=> lam_n = a_n^{3/2}/R

$$an_ := 2.338107411$$

$$lamn := 8.440926174 \times 10^{-23}$$

$$val := -1.080028022 \times 10^{-9}$$

```
  n=1   a_n=2.338107   lam_n=8.44093e-23   Ai(xi(R))=-1.08003e-09
```

$$an_ := 4.087949444$$

$$lamn := 1.951424209 \times 10^{-22}$$

$$val := 6.979273783 \times 10^{-10}$$

```
  n=2   a_n=4.087949   lam_n=1.95142e-22   Ai(xi(R))=6.97927e-10
```

$$an_ := 5.520559828$$

$$lamn := 3.062443571 \times 10^{-22}$$

$$val := 9.478751869 \times 10^{-10}$$

```
  n=3   a_n=5.520560   lam_n=3.06244e-22   Ai(xi(R))=9.47875e-10

=== SECTION 4 : Full field f(r,t) and H(r,theta,t) ===
  f(r,t) = SUM_n (1/r)*Ai(-(lam_n^2/R)^{1/3}*r)*[A_n cos + B_n sin]
  f_1(R,  0) = -2.54993e-32   (BC check ~0)
  f_1(2R, 0) = 4.27065e-24

  H(r,theta,t) = SUM_{n,l} g_n(r)*P_l(cos theta)*[cos/sin]
  P_0=1   P_1=0.707107   P_2=0.25   at theta=Pi/4

=== SECTION 5 : Probability density PDE ===
```

$$PDE_P := \frac{\partial^2}{\partial t^2} P(r,t) - 8.987551787 \times 10^{16} \frac{\partial^2}{\partial r^2} P(r,t) = 0$$

```
  Probability PDE: d^2P/dt^2 - c^2*d^2P/dr^2 = 0
```

$$\frac{\partial^2}{\partial t^2} P(r,t) - 8.987551787 \times 10^{16} \frac{\partial^2}{\partial r^2} P(r,t) = 0$$

```
  d'Alembert: P(r,t) = phi+(r+ct) + phi-(r-ct)
  With P(r,0)=A*r^2+B*r+C and dP/dt|0=0:
  => phi+ = phi- = (A*r^2+B*r+C)/2
  => P(r,t) = A*(r^2+c^2*t^2) + B*r + C   [exact!]

=== SECTION 6 : Analytic determination of A, B, C ===
  P(R,t) = A*c^2*t^2   [using BC: A*R^2+B*R+C=0]
  int(P(R,t),t=t1..t2) = A*c^2*(t2^3-t1^3)/3
  => A = 47Gly / [c^2*(t2^3-t1^3)/3]
```

```
  t2^3-t1^3 = 8.24139e+79 s^3
  (t2^3-t1^3)/3 = 2.74713e+79 s^3
  c^2*(t2^3-t1^3)/3 = 2.46900e+96 m^2*s

  A = 1.80095e-70  m^-1 s^-1
  B = -2*A*R = -1.52559e-47  s^-1
  C = A*R^2  = 3.23084e-25  m s^-1

  VERIFICATION:
  BC: A*R^2+B*R+C = -1e-34  (->0)
  Integral = 4.44654e+26 m
  Target   = 4.44654e+26 m  (47 Gly)
  Ratio    = 1  (->1)
  A != 0 : true
  C != 0 : true

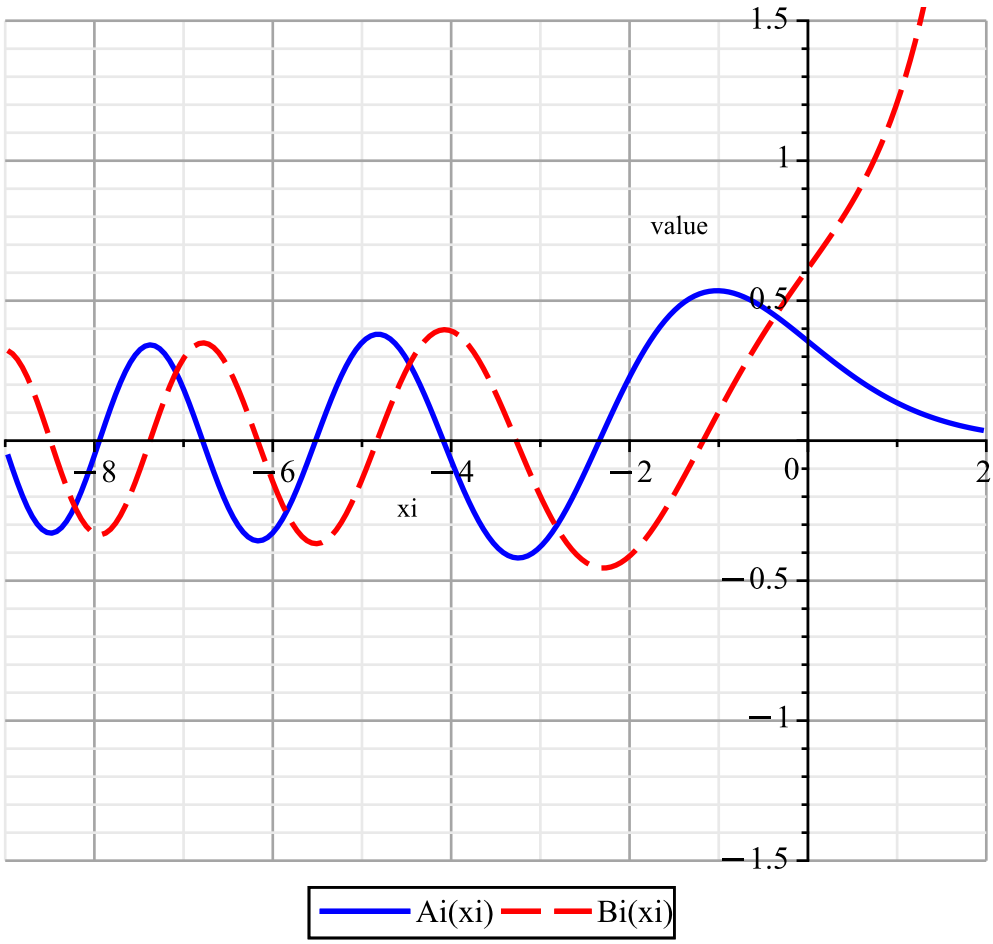
=== SECTION 7 : Explicit P(r,t) and flux F(r,t) ===
  P(r,t) = A*[(r-R)^2 + c^2*t^2]
  F(r,t) = P(r,t)/(4*Pi*r^2)

  P(R,  0)  = 0  (BC check)
  P(2R, 0)  = 3.23084e-25
  P(R, t1)  = 0.00135565 m/s
  P(R, t2)  = 3.06535 m/s
  dP/dt|t=0 = 0  (should be 0)

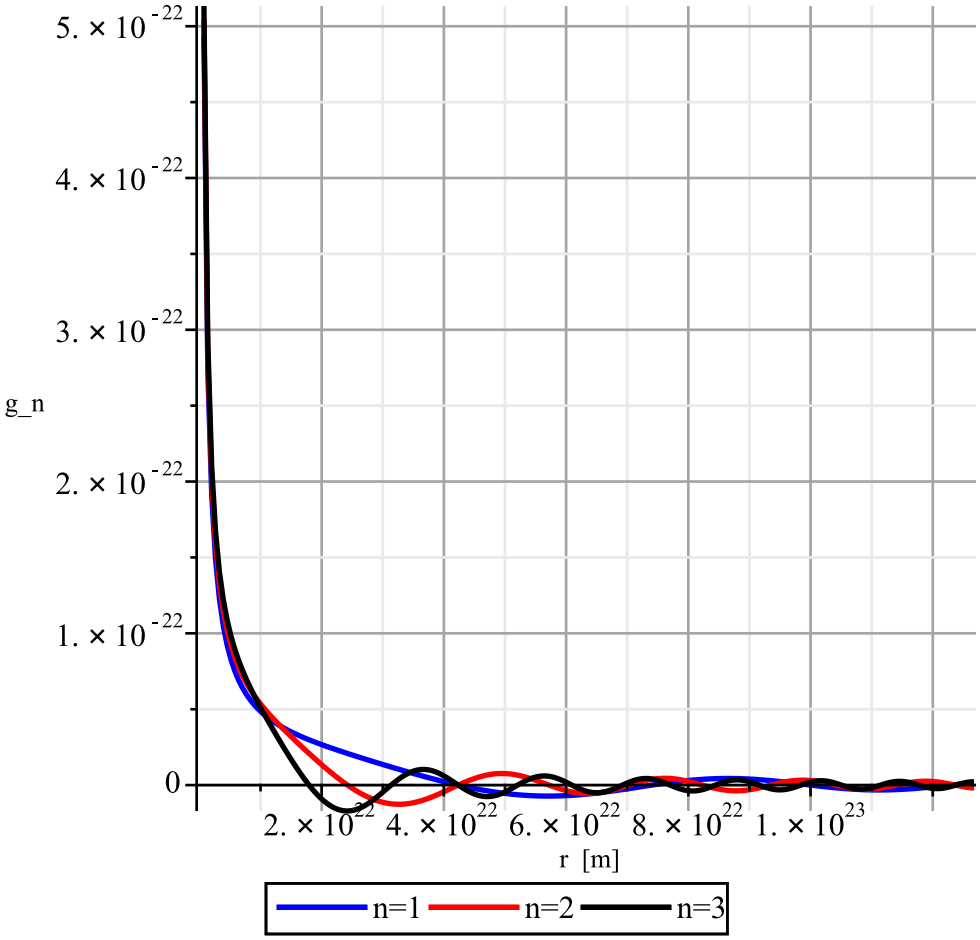
=== SECTION 8 : Generating 12 plots ===
  Plot 1 done
  Plot 2 done
  Plot 3 done
  Plot 4 done
  Plot 5 done
  Plot 6 done
  Plot 7 done
  Plot 8 done
  Plot 9 done
  Plot 10 done
  Plot 11 done
  Plot 12 done

=== Displaying plots 1-12 ===
```

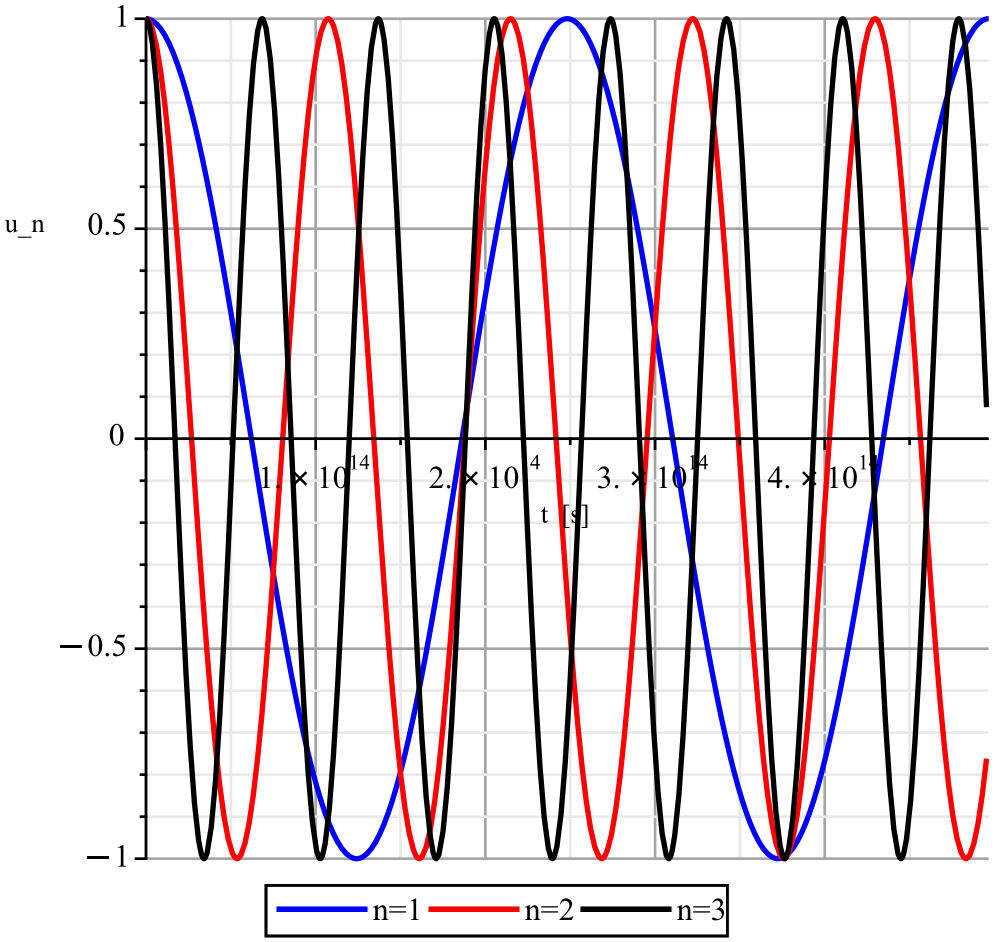
Plot 1 — Airy Functions Ai(xi) and Bi(xi)



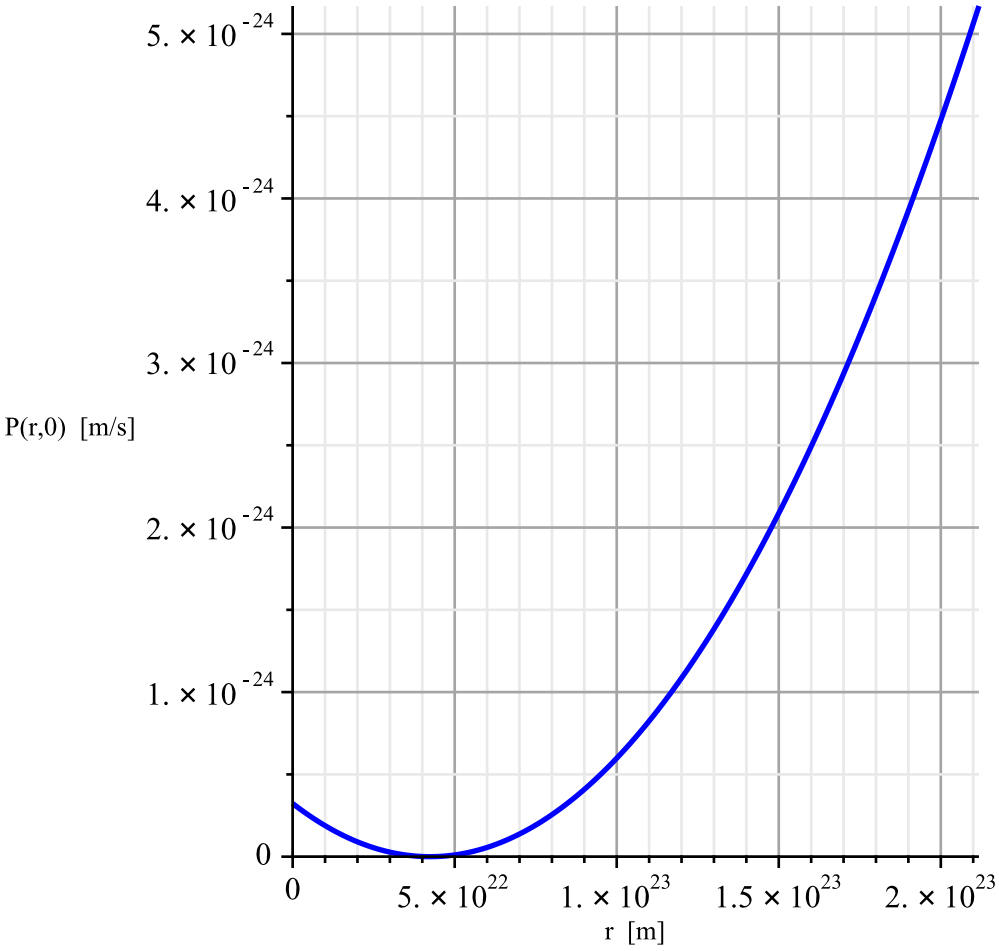
Plot 2 — Radial Eigenfunctions g_n(r) = Ai(xi_n)/r



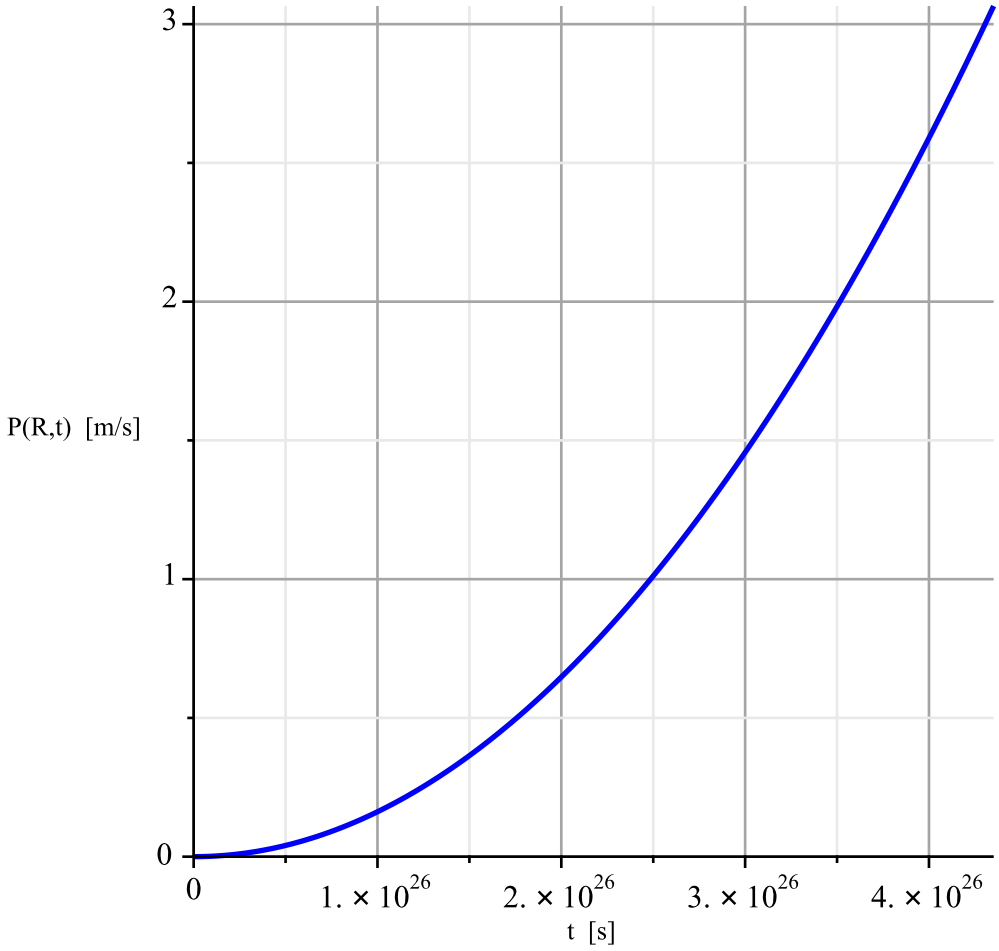
Plot 3 — Temporal Harmonics cos(omega_n*t)



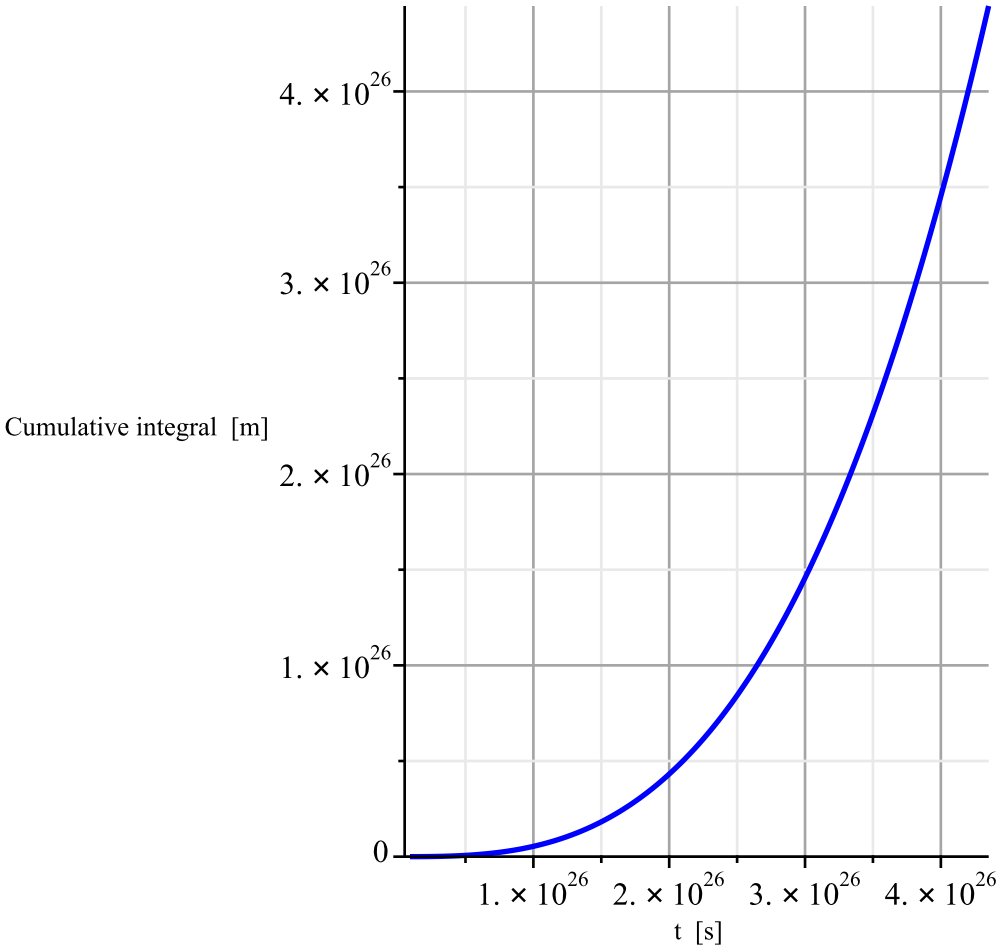
Plot 4 — Initial Density P(r,0) = A*(r-R)^2



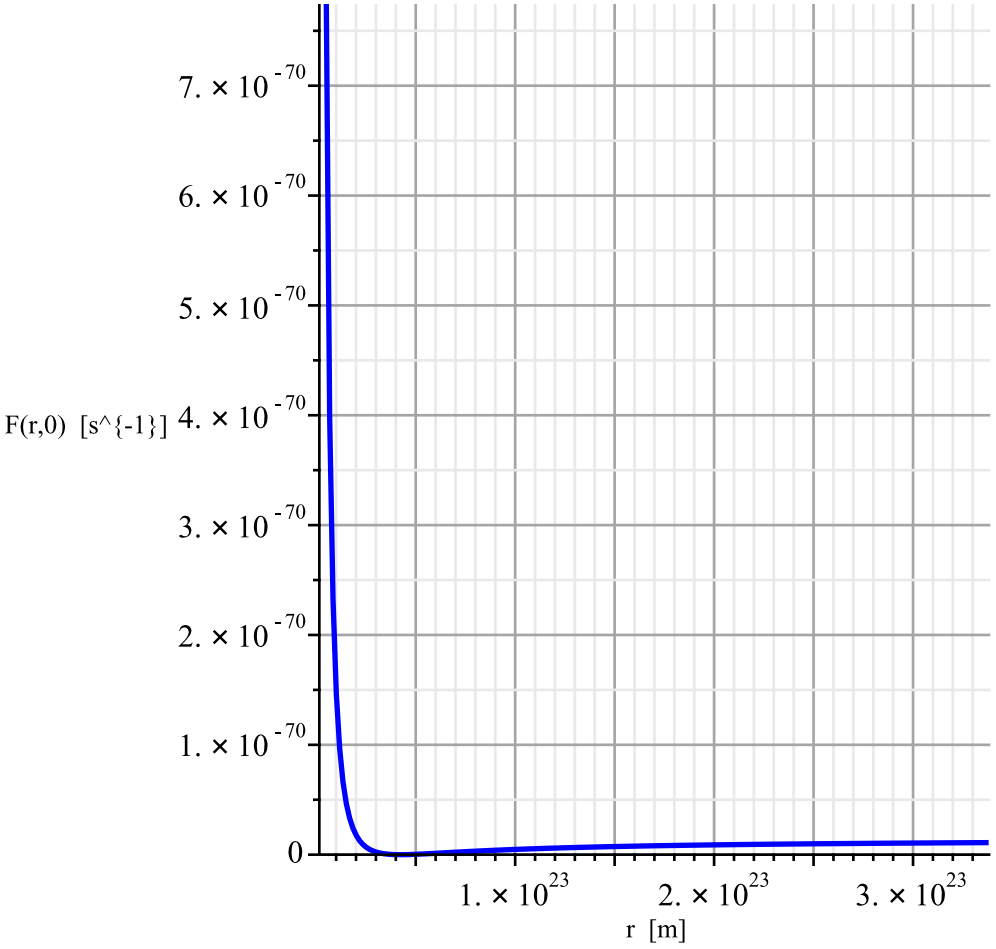
Plot 5 — Density at Boundary P(R,t) = A*c^2*t^2

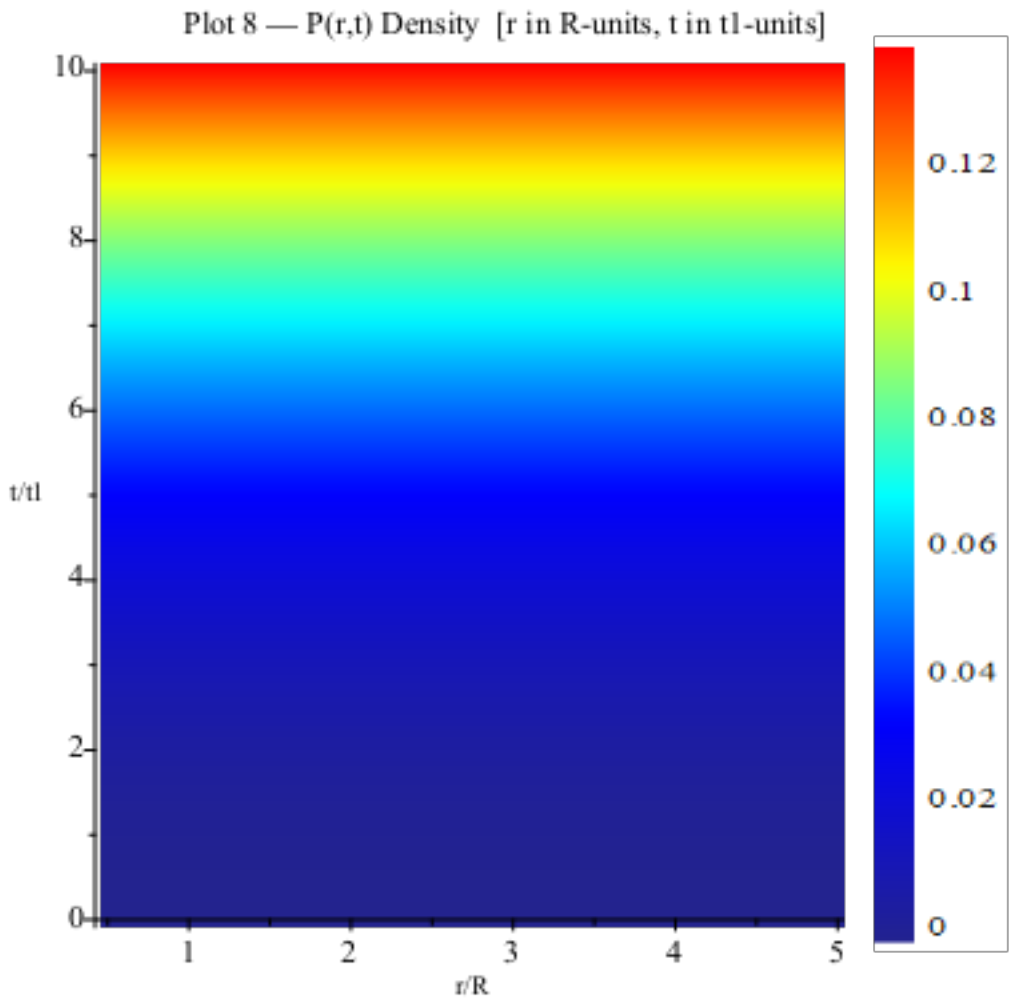


Plot 6 — Cumulative int(P(R,t),t1..t) [m]

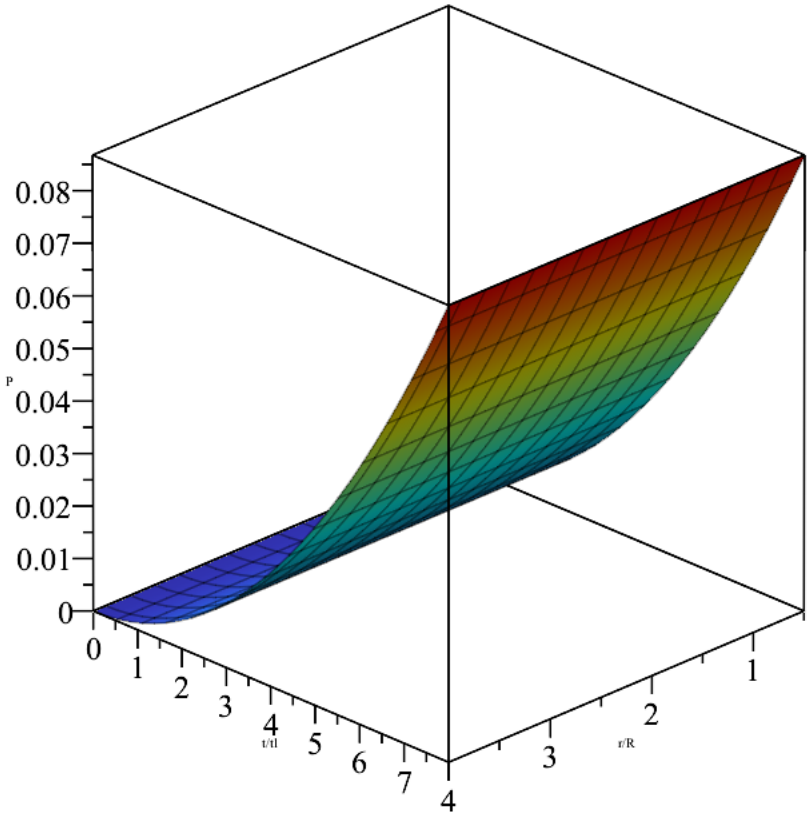


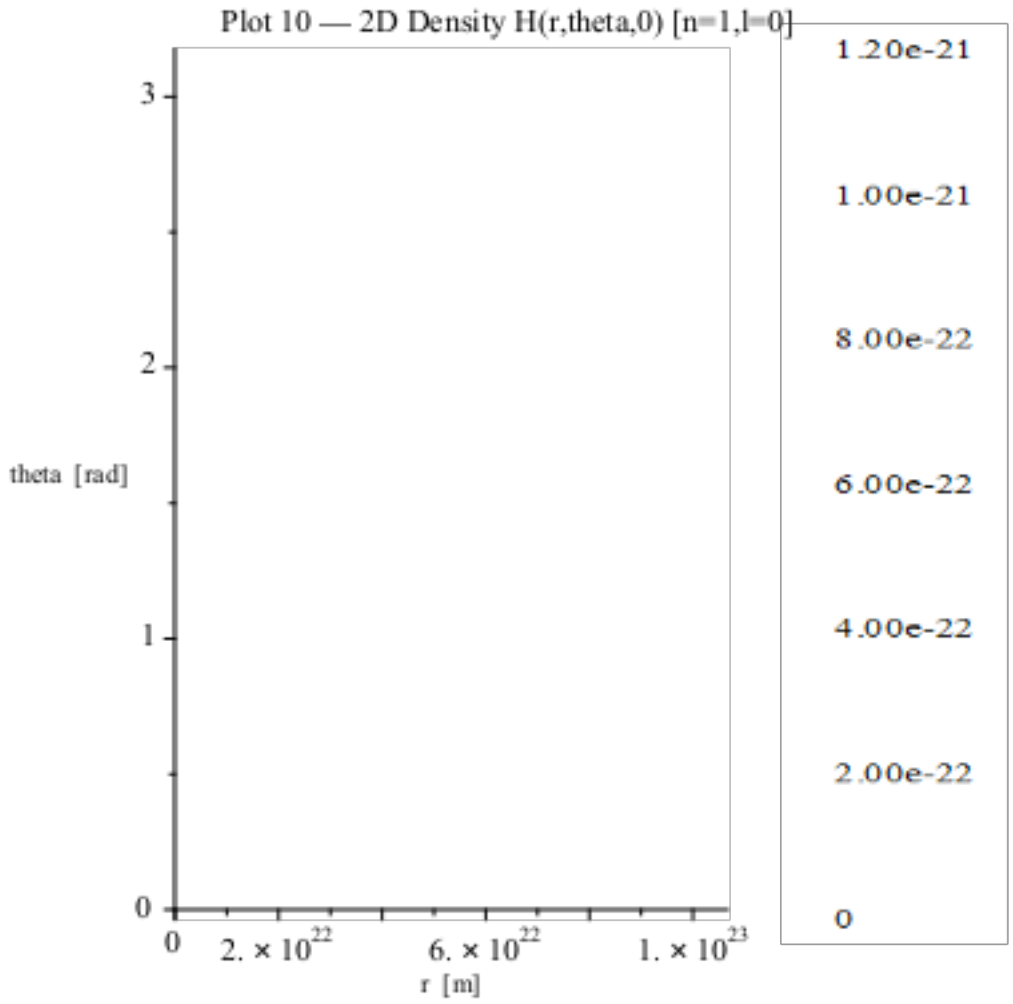
Plot 7 — Flux F(r,0) = P(r,0)/(4*Pi*r^2)



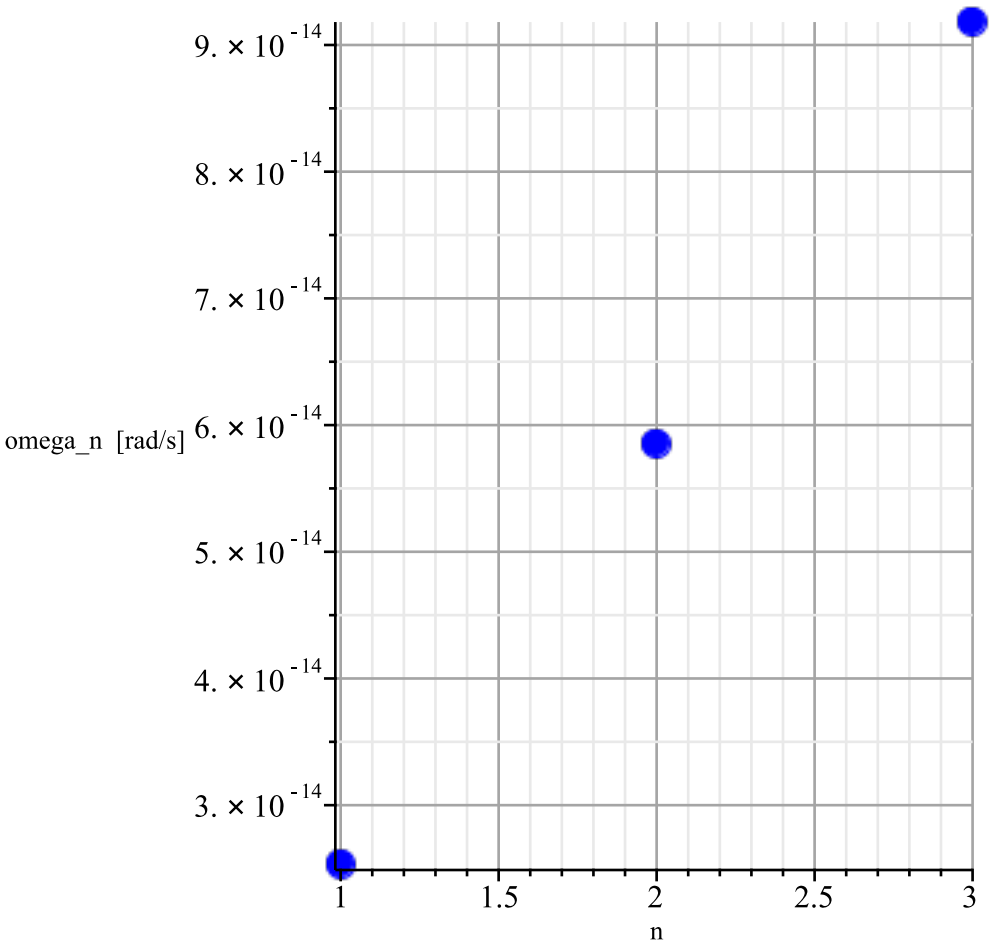

Plot 8 — P(r,t) Density [r in R-units, t in tl-units]
10
8
6
4
2
0
t/tl
1
2
3
4
5
r/R
0.12
0.1
0.08
0.06
0.04
0.02
0

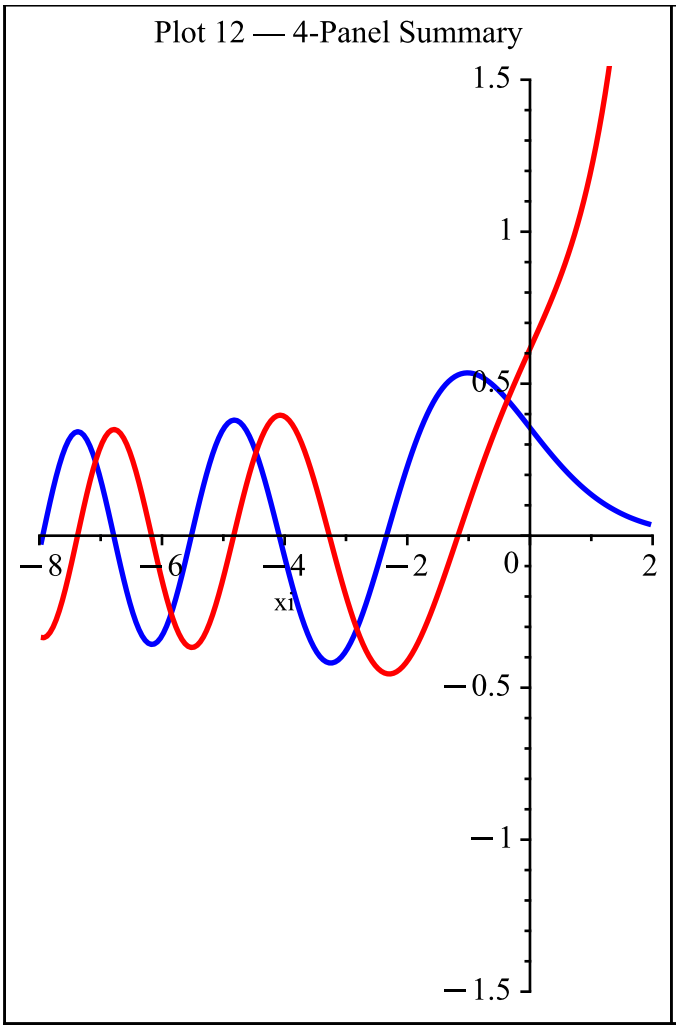
Plot 9 — 3D Surface P(r,t) [r in R, t in t1]

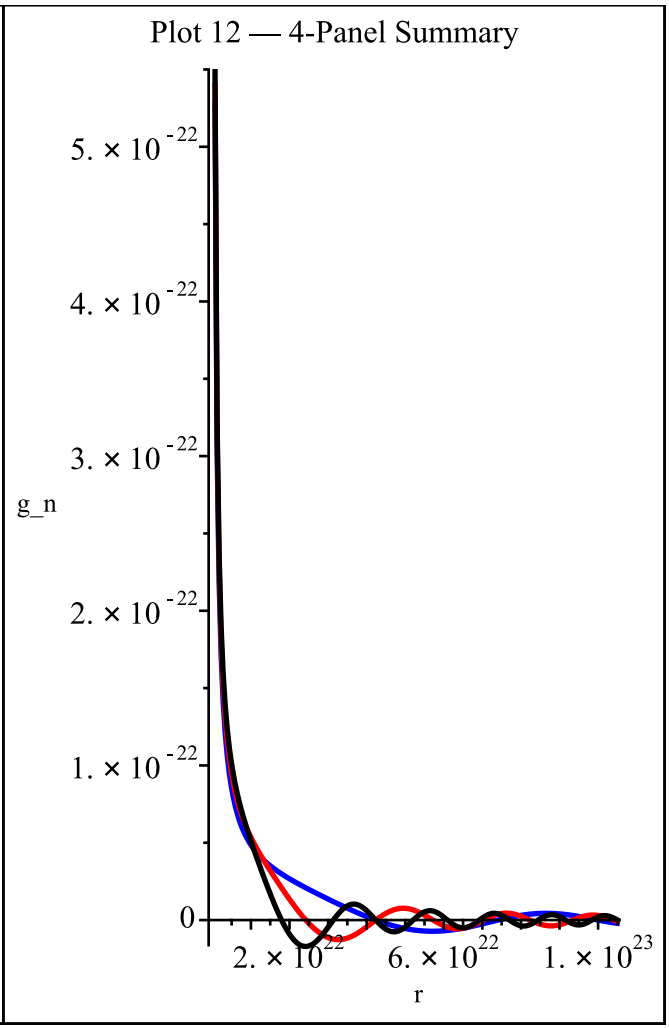


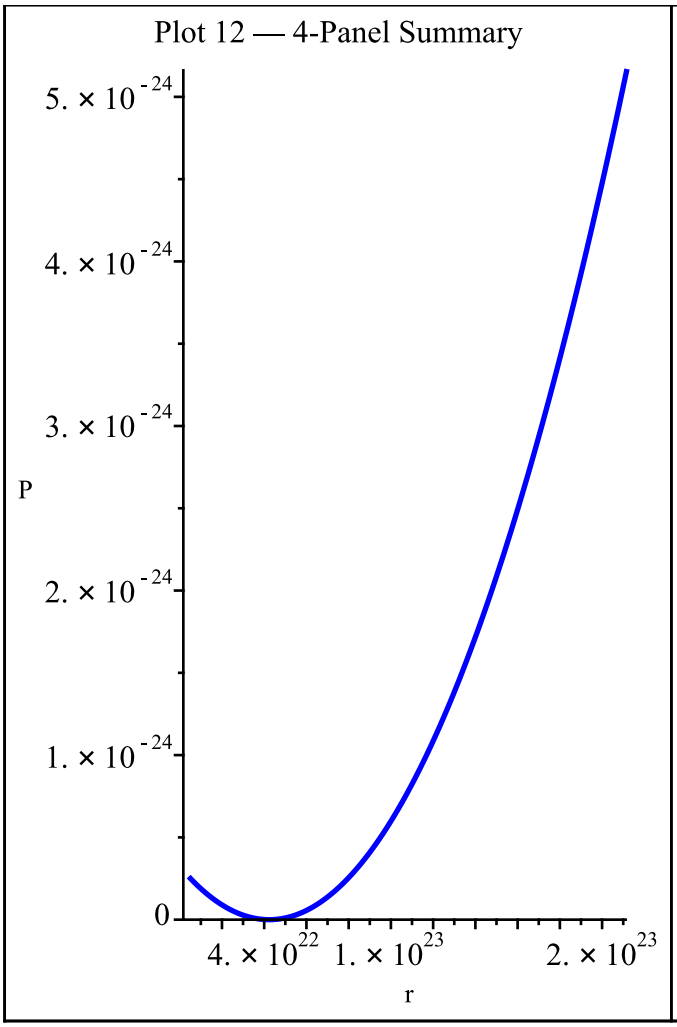

Plot 10 — 2D Density H(r,theta,0) [n=1,l=0]
theta [rad]
r [m]
0
1
2
3
0
2. × 10^22
6. × 10^22
1. × 10^23
1.20e-21
1.00e-21
8.00e-22
6.00e-22
4.00e-22
2.00e-22
0

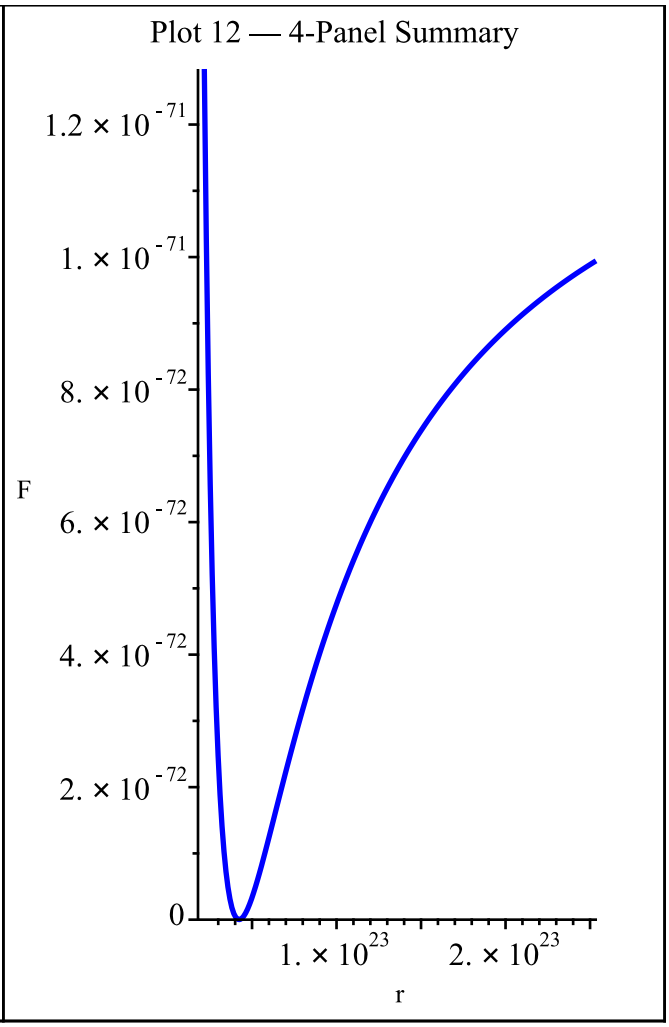
Plot 11 — Eigenfrequencies omega_n vs n




Plot 12 — 4-Panel Summary
1.5
1
0.5
−0.5
−1
−1.5
−8
−6
−4
−2
0
2
xi


Plot 12 — 4-Panel Summary
5. × 10^-22
4. × 10^-22
3. × 10^-22
2. × 10^-22
1. × 10^-22
0
g_n
2. × 10^22
6. × 10^22
1. × 10^23
r






```
=== SECTION 9 : Complete Results Summary ===
-----------------------------------------------------------------------
  Quantity                                  Value
-----------------------------------------------------------------------
  R   (boundary parameter)                  4.23552e+22 m
  c   (speed of light)                      2.99792e+08 m/s
  t1   = 0.290 Gy                           9.15170e+24 s
  t2   = 13.79 Gy                           4.35179e+26 s
  47 Gly                                    4.44654e+26 m
  int(t^2,t1..t2) = (t2^3-t1^3)/3          2.74713e+79 s^3
  c^2*(t2^3-t1^3)/3                         2.46900e+96 m^2*s
  Airy zeros  a1,a2,a3                      2.33811  4.08795  5.52056
  lambda1=a1^{3/2}/R                        8.44093e-23 m^-1
  lambda2=a2^{3/2}/R                        1.95142e-22 m^-1
  lambda3=a3^{3/2}/R                        3.06244e-22 m^-1
  omega1=c*lam1                             2.53053e-14 rad/s
  omega2=c*lam2                             5.85022e-14 rad/s
```

```
  omega3=c*lam3                          9.18097e-14 rad/s
  A = 47Gly/(c^2*(t2^3-t1^3)/3)        1.80095e-70 m^-1 s^-1
  B = -2*A*R                            -1.52559e-47 s^-1
  C = A*R^2                             3.23084e-25 m s^-1
  BC check A*R^2+B*R+C                  -1e-34  (->0)
  Integral A*c^2*(t2^3-t1^3)/3          4.44654e+26 m
  Target 47 Gly                         4.44654e+26 m
  dP/dt|t=0                             0  (rest IC)
  P(r,0) form                           A*(r-R)^2
  P(r,t) form                           A*[(r-R)^2+c^2*t^2]
-------------------------------------------------------------------

All 9 sections and 12 plots completed.
TauTimp_Spherical_Wave_TimeInt.mpl —— Viorel Popescu 2026
```